

Análise de Algoritmos Aplicada à Inserção sem Duplicatas em CSV

Comparação entre busca sequencial iterativa e recursiva: análise
teórica, demonstração por indução e validação experimental

Autores: Vitor Kenzo Matsunaga, Gustavo Alves, Matheus Brunes

Disciplina: Análise de Algoritmos

Professor: Zoe Roberto

Repositório Git público

<https://github.com/owhska/insercao-sem-duplicatas-csv>

Setembro de 2026

Sumário

1	Introdução	2
2	Descrição da solução implementada	2
3	Explicação do funcionamento do programa	3
4	Estruturas de dados utilizadas	3
5	Análise teórica do algoritmo	4
5.1	Hipóteses	4
5.2	Custo de uma inserção isolada	4
5.3	Complexidade assintótica	4
5.4	Comportamento quando N dobra	4
6	Dedução da função $T(N, Q)$	5
7	Demonstração por indução matemática	5
8	Análise das duas implementações da busca sequencial	6
8.1	Busca sequencial iterativa	6
8.2	Busca sequencial recursiva	6
8.3	Comparação teórica entre iterativa e recursiva	7
8.4	Custo total do programa	7
9	Metodologia experimental	7
9.1	Ambiente	7
9.2	Preparação das entradas	7
9.3	Os cinco experimentos	8
9.4	Execução em ambos os modos	8
10	Tabelas e gráficos dos experimentos	8
10.1	Experimento A	8
10.2	Experimento B	9
10.3	Experimento C	9
10.4	Gráficos	10
10.5	Comparação iterativa $\times$ recursiva (tempo)	10
10.6	Experimentos D e E	10
11	Comparação entre os resultados teóricos e empíricos	11
11.1	Aderência quantitativa	11
11.2	O exemplo do enunciado como caso particular $Q = 1$	11
11.3	Taxa de crescimento ao dobrar N	12
11.4	Comparação de tempo entre iterativa e recursiva	12
11.5	O desvio do Experimento D, e por que ele é esperado	13
12	Discussão dos resultados	14
13	Conclusão	15
14	Repositório Git	15

1 Introdução

Este trabalho tem por objetivo implementar, em linguagem C, um serviço de importação de registros de bolsistas a partir de arquivos CSV, garantindo que nenhum nome seja cadastrado em duplicidade, e analisar rigorosamente o custo computacional do algoritmo empregado na verificação de redundância. A detecção de duplicatas é feita por busca sequencial sobre os nomes já presentes no cadastro de destino, e o programa foi instrumentado com um contador do número de comparações entre nomes realizado durante a execução.

Além da implementação, este relatório compara duas formas de codificar a mesma busca sequencial — uma iterativa e outra recursiva — discutindo suas diferenças de custo em tempo, espaço auxiliar e risco de estouro de pilha, e demonstra que ambas produzem exatamente o mesmo número de comparações para qualquer entrada.

O objetivo central não é apenas mostrar que o programa funciona, mas confrontar uma previsão teórica com evidências medidas. A análise parte de dois parâmetros: Q , a quantidade de cadastros já existentes no arquivo de destino no início da execução, e N , a quantidade de nomes novos e distintos a inserir. A função de custo deduzida é

$$T(N, Q) = N \cdot Q + \frac{N(N-1)}{2}, \quad (1)$$

que é demonstrada formalmente por indução matemática e depois submetida a validação experimental. Os experimentos foram conduzidos sobre o arquivo `bolsistas-2025.csv`, obtido do Portal de Dados Abertos (dados.gov.br), que contém 719 registros e 718 nomes distintos, variando N de 10 a 718 e Q de 0 a 160, em ambos os modos de busca (iterativo e recursivo).

Adianta-se o resultado principal: em todas as execuções em que a hipótese de nomes distintos é satisfeita, a diferença entre o número de comparações previsto pela Equação (1) e o número medido pelo contador foi exatamente zero, tanto no modo iterativo quanto no recursivo.

2 Descrição da solução implementada

O programa é um único executável em C, copiador, invocado da seguinte forma:

```
./copiador <origem.csv> <destino.csv> <estatisticas.csv> [iter|rec]
```

`origem.csv` arquivo de onde os registros são lidos;

`destino.csv`

cadastro principal, para onde os registros não redundantes são copiados. É criado com o cabeçalho caso não exista;

`estatisticas.csv`

arquivo onde são registrados, ao final, os nomes inseridos, o total de comparações e o tempo de execução;

`[iter|rec]` parâmetro opcional que seleciona o modo de busca: iterativo (padrão) ou recursivo.

A estratégia de solução é a mais direta possível, e essa escolha é deliberada: o objetivo do exercício é analisar o comportamento da busca sequencial, e não otimizá-lo. O programa mantém em memória um vetor com os nomes já presentes no destino e, para cada registro lido da origem, percorre esse vetor comparando nome a nome até encontrar uma igualdade ou esgotar o vetor.

Três decisões de implementação merecem registro. Primeiro, o destino é carregado por completo antes de começar a cópia, o que estabelece o valor de Q ; sem essa etapa, o programa reinseriria nomes já cadastrados a cada nova execução. Segundo, a comparação considera exclusivamente o campo Nome: dois registros com o mesmo nome são redundantes mesmo que Modalidade, Nivel e Agencia sejam diferentes. Terceiro, uma redundância não interrompe a execução — ao encontrar um nome repetido, o programa emite uma mensagem de erro e segue para o próximo registro.

3 Explicação do funcionamento do programa

O fluxo de execução é o seguinte:

1. **Validação dos argumentos.** Se o número de argumentos estiver fora do esperado, o programa imprime a forma de uso em `stderr` e encerra com código de erro.
2. **Preparação do destino.** A função `garantir_cabecalho` abre o arquivo de destino em modo `a+` e, se estiver vazio, grava a linha de cabeçalho `Nome, Modalidade, Nivel, Agencia`.
3. **Carga do cadastro existente.** A função `carregar_destino` lê o destino linha a linha, ignora cabeçalho e linhas vazias, extrai o campo `Nome` e o insere no vetor em memória. O valor retornado é Q .
4. **Laço principal.** Cada linha da origem é lida; extrai-se o campo `Nome` e a função `busca_sequencial` (despachada para a versão iterativa ou recursiva, conforme o modo escolhido) percorre o vetor em busca dele.
5. **Decisão.** Se a busca encontra o nome, uma mensagem de erro é emitida e o registro é descartado; caso contrário, a linha é gravada no destino e o nome passa a integrar o vetor.
6. **Relatório e estatísticas.** Ao final, o programa imprime um resumo na saída padrão e acrescenta ao arquivo de estatísticas uma linha com nomes inseridos, comparações e tempo de execução.

A instrumentação está concentrada em um único ponto: o contador global `comparacoes` é incrementado exatamente uma vez por chamada a `strcmp`, tanto na versão iterativa quanto na recursiva (Seção 8). Essa é a definição operacional de comparação adotada neste trabalho: uma comparação é cada operação em que o `Nome` de um registro lido da origem é confrontado com o `Nome` de um registro já presente no destino.

4 Estruturas de dados utilizadas

A estrutura central é um vetor dinâmico de cadeias de caracteres, que representa o conjunto de nomes presentes no destino:

Listing 1: *Estrutura de dados Cadastro*

```
typedef struct {
    char **nomes;
    size_t qtd;
    size_t capacidade;
} Cadastro;
```

O campo `nomes` é um vetor de ponteiros; cada posição aponta para uma cadeia alocada individualmente com o tamanho exato do nome. O campo `qtd` guarda quantos nomes estão armazenados, e `capacidade`, quantos caberiam sem realocação.

Crescimento amortizado. Quando `qtd` alcança `capacidade`, a função `cadastro_inserir` dobra a capacidade com `realloc` (16 na primeira alocação). O dobramento garante que o custo total das realocações ao longo de N inserções seja $O(N)$, isto é, $O(1)$ amortizado por inserção. O crescimento do vetor, portanto, não contribui para o termo quadrático do custo total, que vem inteiramente das comparações.

Por que um vetor, e não uma tabela de dispersão. Uma tabela hash ou uma árvore de busca balanceada resolveria o mesmo problema com custo total $O(N)$ ou $O(N \log N)$ em vez de $O(N^2)$. A escolha do vetor com busca sequencial é imposta pelo enunciado, precisamente porque é o cenário cujo custo se quer medir.

5 Análise teórica do algoritmo

5.1 Hipóteses

A análise adota as seguintes hipóteses:

(H1) a verificação de redundância é feita por busca sequencial;

(H2) os N nomes a serem inseridos são distintos entre si;

(H3) o arquivo de destino inicia com Q cadastros.

A hipótese (H2) é a mais importante, pois torna a análise determinística: se todos os nomes são distintos e nenhum deles já está no destino, nenhuma busca encontra o que procura, e portanto toda busca percorre o vetor até o fim. Não existe caso médio nem melhor caso a considerar — há um único comportamento possível, que é o pior caso da busca sequencial.

5.2 Custo de uma inserção isolada

Considere a i -ésima inserção, com $i = 1, 2, \dots, N$. Antes dela, o vetor contém os Q nomes que já estavam no destino e os $i - 1$ nomes inseridos pelas iterações anteriores, totalizando $Q + i - 1$ elementos. Logo, a busca malsucedida executa

$$c_i = Q + i - 1 \quad (2)$$

comparações. O custo de cada inserção cresce linearmente com o número de inserções já feitas, porque o vetor contra o qual se compara cresce a cada passo.

5.3 Complexidade assintótica

Expandindo a Equação (1):

$$T(N, Q) = \frac{N^2}{2} + N \left(Q - \frac{1}{2} \right).$$

Para Q constante, o termo dominante é $N^2/2$, portanto $T(N) = O(N^2)$; de fato, $T(N) = \Theta(N^2)$. Para o limite superior, tomando $N \geq \max(Q, 2)$, tem-se $NQ \leq N^2$ e $N(N-1)/2 \leq N^2/2$, logo

$$T(N, Q) \leq N^2 + \frac{N^2}{2} = \frac{3}{2}N^2,$$

o que satisfaz $O(N^2)$ com $c = 3/2$ e $n_0 = \max(Q, 2)$. Para o limite inferior, para $N \geq 2$ vale $N-1 \geq N/2$, logo

$$T(N, Q) \geq \frac{N(N-1)}{2} \geq \frac{N^2}{4},$$

o que satisfaz $\Omega(N^2)$ com $c = 1/4$ e $n_0 = 2$. Das duas condições segue $T(N) = \Theta(N^2)$. Essa classificação não muda se Q crescer proporcionalmente a N : se $Q = \alpha N$, então $T = (\alpha + 1/2)N^2 - N/2$, que continua sendo $\Theta(N^2)$, apenas com uma constante multiplicativa maior.

5.4 Comportamento quando N dobra

Com $Q = 0$, a razão entre o custo de $2N$ e o de N nomes é

$$\frac{T(2N)}{T(N)} = \frac{2N(2N-1)}{N(N-1)} = \frac{4N-2}{N-1}. \quad (3)$$

Tomando o limite, $\lim_{N \rightarrow \infty} \frac{4N-2}{N-1} = 4$: ao dobrar a quantidade de nomes, o número de comparações quadruplica. A convergência se dá por cima, e a distância até 4, que vale $\frac{2}{N-1}$, cai pela metade a cada duplicação de N . O termo NQ , quando presente, não altera esse limite: para Q fixo ele é linear em N e, portanto, assintoticamente desprezível diante do termo quadrático; seu efeito é apenas retardar a convergência da razão para 4.

6 Dedução da função $T(N, Q)$

O total de comparações é a soma dos custos individuais das N inserções. Usando a Equação (2):

$$\begin{aligned}
 T(N, Q) &= \sum_{i=1}^N c_i = \sum_{i=1}^N (Q + i - 1) \\
 &= \sum_{i=1}^N Q + \sum_{i=1}^N (i - 1) \\
 &= N \cdot Q + \sum_{j=0}^{N-1} j \quad (j = i - 1) \\
 &= N \cdot Q + \frac{(N-1)N}{2} \\
 &= N \cdot Q + \frac{N(N-1)}{2}.
 \end{aligned} \tag{4}$$

A fórmula tem uma leitura direta: o termo $N \cdot Q$ é o custo de comparar cada um dos N nomes novos contra os Q cadastros que já estavam no destino — um retângulo de N por Q comparações — e o termo $N(N-1)/2$ é o custo de comparar os nomes novos entre si: são $\binom{N}{2}$ pares não ordenados, cada um comparado exatamente uma vez.

Casos particulares. Para $Q = 0$, a fórmula se reduz a $T(N, 0) = N(N-1)/2$. Para $Q = 1$, ocorre uma simplificação notável:

$$T(N, 1) = N + \frac{N(N-1)}{2} = \frac{N(N+1)}{2}. \tag{4}$$

Esse caso é importante porque é exatamente ele que reproduz o exemplo de saída apresentado no enunciado (Seção 11).

7 Demonstração por indução matemática

Demonstra-se, por indução sobre N , a validade da Equação (1). Seja $Q \geq 0$ fixo e defina-se a proposição

$$P(N) : \quad T(N, Q) = N \cdot Q + \frac{N(N-1)}{2}.$$

Caso base. Para $N = 1$, há uma única inserção; no momento em que ela ocorre, o vetor contém apenas os Q cadastros iniciais, e a busca — malsucedida, por (H2) — percorre todos eles. Logo $T(1, Q) = Q$. Avaliando a fórmula em $N = 1$: $1 \cdot Q + \frac{1 \cdot 0}{2} = Q$. Os dois valores coincidem, logo $P(1)$ é verdadeira. (A fórmula também vale trivialmente para $N = 0$, o que permite ancorar a indução em $N = 0$ caso se prefira.)

Hipótese de indução. Suponha que, para algum $k \geq 1$, $P(k)$ seja verdadeira, isto é, $T(k, Q) = kQ + \frac{k(k-1)}{2}$.

Passo indutivo. Considere a inserção de $k+1$ nomes distintos. As k primeiras inserções são idênticas às do caso com k nomes, custando, por hipótese de indução, $T(k, Q)$. Resta o custo da $(k+1)$ -ésima inserção: o vetor contém então $Q+k$ elementos, e como o novo nome é distinto de todos eles,

por (H2), a busca percorre o vetor inteiro, custando $Q + k$ comparações. Logo:

$$\begin{aligned}
 T(k+1, Q) &= T(k, Q) + (Q + k) \\
 &= kQ + \frac{k(k-1)}{2} + Q + k \\
 &= (k+1)Q + \frac{k(k-1) + 2k}{2} \\
 &= (k+1)Q + \frac{k^2 + k}{2} \\
 &= (k+1)Q + \frac{(k+1)k}{2} \\
 &= (k+1)Q + \frac{(k+1)((k+1)-1)}{2},
 \end{aligned}$$

que é precisamente a fórmula avaliada em $N = k + 1$. Portanto $P(k + 1)$ é verdadeira.

Conclusão. Verificou-se que $P(1)$ é verdadeira e que, para todo $k \geq 1$, $P(k) \Rightarrow P(k + 1)$. Pelo princípio da indução matemática,

$$T(N, Q) = N \cdot Q + \frac{N(N-1)}{2} \quad \text{para todo } N \geq 1,$$

sob as hipóteses (H1), (H2) e (H3). ■

Cabe sublinhar o que a demonstração estabelece e o que não estabelece: ela prova que a fórmula descreve exatamente o número de comparações quando os nomes inseridos são distintos, mas nada afirma sobre execuções com redundâncias — nesses casos, uma busca pode terminar antes de percorrer o vetor todo. O Experimento D (Seção 10.6) explora justamente essa fronteira.

8 Análise das duas implementações da busca sequencial

8.1 Busca sequencial iterativa

Listing 2: Busca sequencial iterativa

```

1 int busca_sequencial_iterativa(const Cadastro *c, const char *nome) {
2     size_t i;
3     for (i = 0; i < c->qtd; i++) {
4         comparacoes++;
5         if (strcmp(c->nomes[i], nome) == 0) return 1;
6     }
7     return 0;
8 }

```

Análise. Tempo no pior caso $\Theta(n)$; no melhor caso, $\Theta(1)$; no caso médio, $\Theta(n)$. Espaço auxiliar $\Theta(1)$: apenas a variável de controle i é usada, independentemente do tamanho do vetor. Espaço total $\Theta(n)$, correspondente ao próprio cadastro.

8.2 Busca sequencial recursiva

Listing 3: Busca sequencial recursiva

```

1 int busca_sequencial_recursiva_aux(const Cadastro *c, const char *nome,
2                                     size_t i) {
3     if (i >= c->qtd) return 0;
4     comparacoes++;
5     if (strcmp(c->nomes[i], nome) == 0) return 1;
6     return busca_sequencial_recursiva_aux(c, nome, i + 1);
7 }
8
9 int busca_sequencial_recursiva(const Cadastro *c, const char *nome) {
10     return busca_sequencial_recursiva_aux(c, nome, 0);
11 }

```

Análise. As complexidades de tempo são idênticas às da versão iterativa: $\Theta(n)$ no pior e no caso médio, $\Theta(1)$ no melhor caso. A diferença está no espaço: cada chamada recursiva empilha uma nova moldura de ativação, de modo que o espaço auxiliar é $\Theta(n)$ — uma moldura por elemento do vetor no pior caso — e o espaço total permanece $\Theta(n) + \Theta(n) = \Theta(n)$, mas com uma constante maior que a versão iterativa.

8.3 Comparação teórica entre iterativa e recursiva

Tabela 1: Comparação teórica entre as implementações iterativa e recursiva

Critério	Iterativa	Recursiva
Tempo pior caso	$\Theta(n)$	$\Theta(n)$
Tempo melhor caso	$\Theta(1)$	$\Theta(1)$
Espaço auxiliar	$\Theta(1)$	$\Theta(n)$
Risco de stack overflow	Não	Sim (n grande)
Overhead por chamada	Nenhum	Custo de chamada
Legibilidade	Alta	Média

Ambas têm a mesma complexidade temporal assintótica $\Theta(n)$, mas a iterativa é espacialmente superior ($\Theta(1)$ contra $\Theta(n)$), sem o risco de estouro de pilha em cadastros muito grandes.

8.4 Custo total do programa

Como as duas implementações incrementam o contador global `comparacoes` exatamente uma vez por chamada a `strcmp`, e percorrem o vetor na mesma ordem, parando no mesmo ponto, o custo total do programa — medido em número de comparações — é idêntico nos dois modos e segue a mesma função $T(N, Q)$ deduzida na Seção 6. A diferença entre os modos manifesta-se apenas no tempo de parede de execução, devido ao overhead de chamada de função da recursão, e no consumo de pilha, não no número de operações lógicas realizadas (Seção 11.4).

9 Metodologia experimental

9.1 Ambiente

Os experimentos foram executados em ambiente Windows 11 x86-64 (GCC 13.3.0, via MSYS2) e em Linux x86-64, com compilação por

```
gcc -O2 -Wall -Wextra -std=c99 -o src/copiador src/copiador.c
```

O nível de otimização é irrelevante para a contagem de comparações: trata-se de uma contagem de eventos discretos do algoritmo, e não de um intervalo de tempo. Essa propriedade é retomada na discussão (Seção 12).

9.2 Preparação das entradas

O arquivo `bolsistas-2025.csv` contém 719 registros e 718 nomes distintos — há exatamente uma redundância real, o nome MARCELO MACHADO VIANA, que aparece duas vezes com Modalidade, Nível e Agência diferentes. Para os experimentos que validam a fórmula, foram gerados recortes contendo os N primeiros nomes distintos do arquivo, com $N \in \{10, 20, 40, 80, 160, 320, 640, 718\}$, o que assegura o cumprimento da hipótese (H2). Os cinco primeiros valores são os exigidos pelo enunciado; os três últimos estendem a curva e permitem observar a convergência da razão de crescimento.

9.3 Os cinco experimentos

Experimento A ($Q = 0$) Destino iniciando vazio, para os oito valores de N . Testa o termo quadrático isoladamente. Previsão: $N(N-1)/2$.

Experimento B ($Q = 1$) Mesmos recortes, destino iniciando com um cadastro prévio. Previsão: $N(N+1)/2$.

Experimento C (Q variável) $N = 160$ fixo, com $Q \in \{0, 10, 20, 40, 80, 160\}$. Isola o termo $N \cdot Q$: com N constante, a fórmula prevê dependência linear em Q , com inclinação 160.

Experimento D (arquivo completo) Origem igual ao `bolsistas-2025.csv` integral, com a redundância real presente, e destino vazio. A hipótese (H2) é violada de propósito.

Experimento E (reexecução) O programa é executado novamente sobre o destino já preenchido pelo Experimento D. Todos os 719 registros são redundantes e nada é inserido.

9.4 Execução em ambos os modos

Cada um dos cinco experimentos foi executado nos dois modos de busca (iterativo e recursivo), totalizando 44 execuções sob a hipótese (H2) — 22 por experimento A/B/C, repetidas em cada modo — mais 4 execuções fora dela (D e E, em cada modo). Adicionalmente, para a comparação de tempo de execução da Seção 11.4, o Experimento A foi repetido 10 vezes em cada ponto e em cada modo, e os tempos foram tomados pela média dessas 10 execuções (Seção 11.4).

10 Tabelas e gráficos dos experimentos

10.1 Experimento A

Tabela 2: Experimento A ($Q = 0$): comparações iterativa e recursiva

N	Comp. iter.	Comp. rec.	Previsto $T(N, 0)$	Diferença
10	45	45	45	0
20	190	190	190	0
40	780	780	780	0
80	3.160	3.160	3.160	0
160	12.720	12.720	12.720	0
320	51.040	51.040	51.040	0
640	204.480	204.480	204.480	0
718	257.403	257.403	257.403	0

Os cinco primeiros valores de N são os exigidos pelo enunciado; os três últimos estendem a curva. A diferença é nula em todos os pontos, em ambos os modos de busca.

10.2 Experimento B

Tabela 3: Experimento B ($Q = 1$): comparações iterativa e recursiva

N	Comp. iter.	Comp. rec.	Previsto $T(N, 1)$	Diferença
10	55	55	55	0
20	210	210	210	0
40	820	820	820	0
80	3.240	3.240	3.240	0
160	12.880	12.880	12.880	0
320	51.360	51.360	51.360	0
640	205.120	205.120	205.120	0
718	258.121	258.121	258.121	0

Os cinco primeiros valores reproduzem exatamente o exemplo de saída do enunciado, em ambos os modos.

10.3 Experimento C

Tabela 4: Experimento C ($N = 160$ fixo): isolamento do termo $N \cdot Q$

Q	Comp. iter.	Comp. rec.	Previsto $T(160, Q)$	Diferença
0	12.720	12.720	12.720	0
10	14.320	14.320	14.320	0
20	15.920	15.920	15.920	0
40	19.120	19.120	19.120	0
80	25.520	25.520	25.520	0
160	38.320	38.320	38.320	0

Cada incremento de 10 em Q acrescenta exatamente 1.600 comparações, isto é, 160×10 , em ambos os modos.

10.4 Gráficos

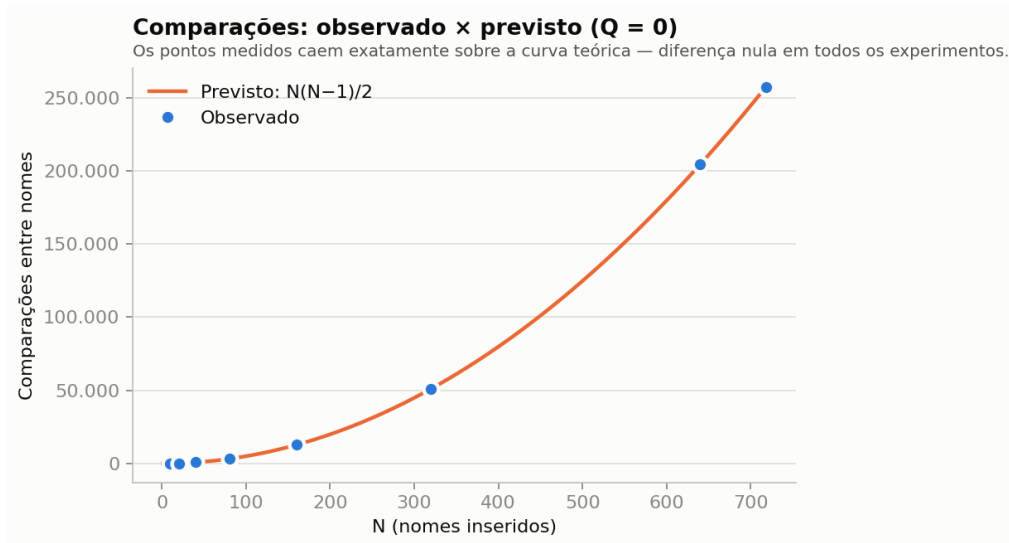


Figura 1: Comparações observadas (pontos) sobre a curva teórica $N(N-1)/2$ (linha) — Experimento A. Não há ponto fora da curva: a sobreposição é exata, não aproximada.

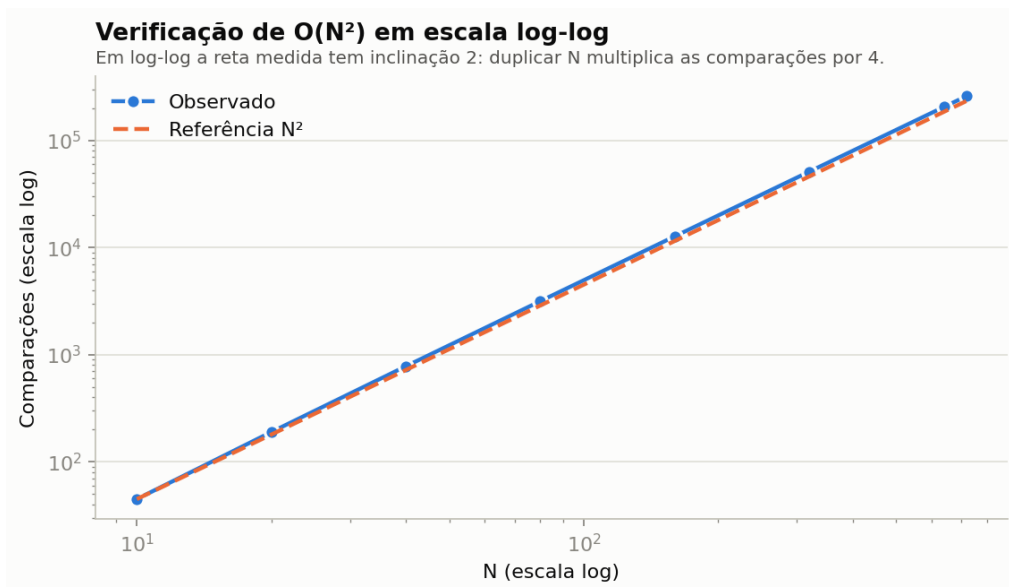


Figura 2: Os mesmos dados em escala log-log. O ajuste por mínimos quadrados sobre os dados medidos resulta em inclinação 2,0198, coerente com $\Theta(N^2)$.

10.5 Comparação iterativa × recursiva (tempo)

Ver Seção 11.4, adiante, que apresenta os tempos médios medidos.

10.6 Experimentos D e E

O Experimento D insere os mesmos 718 nomes do último ponto do Experimento A, mas registra 438 comparações a mais, em ambos os modos de busca (a recursividade não altera o número de comparações, apenas a forma como elas são contadas na pilha de chamadas).

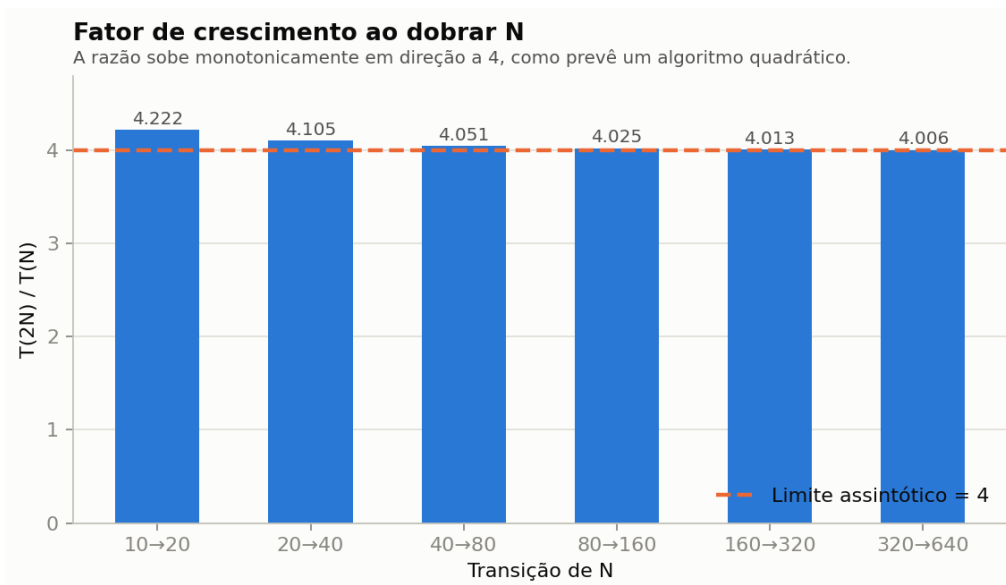


Figura 3: Razão $T(2N)/T(N)$ medida em cada duplicação de N . A razão decresce monotonicamente em direção a 4, exatamente como prevê a Equação (3).

Tabela 5: Experimentos com redundâncias reais (fora da hipótese H2)

Experimento	Registros lidos	Inseridos	Redundantes	Comparações
D — arquivo completo, $Q = 0$	719	718	1	257.841
E — reexecução, $Q = 718$	719	0	719	258.559

11 Comparação entre os resultados teóricos e empíricos

11.1 Aderência quantitativa

Nos Experimentos A, B e C — as 22 execuções em que a hipótese (H2) é satisfeita, replicadas em ambos os modos de busca — a diferença entre observado e previsto foi zero em todos os pontos, sem exceção, tanto no modo iterativo quanto no recursivo. Não se trata de uma aproximação com erro pequeno, mas de coincidência exata dígito a dígito, inclusive em valores da ordem de $2,5 \times 10^5$ comparações.

Três aspectos distintos da fórmula foram testados separadamente: o termo quadrático $N(N-1)/2$ (Experimento A); o termo linear $N \cdot Q$ (Experimento C, mantendo N fixo e variando Q : cada acréscimo de 10 cadastros prévios produziu exatamente 1.600 comparações adicionais); e a combinação dos dois (Experimento B).

11.2 O exemplo do enunciado como caso particular $Q = 1$

O exemplo de saída fornecido no enunciado — os pares N /comparações (10, 55), (20, 210), (40, 820), (80, 3240), (160, 12880) — não corresponde a $N(N-1)/2$ (que daria 45, 190, 780, 3.160 e 12.720), mas sim a $N(N+1)/2$, que é precisamente a especialização de $T(N, Q)$ em $Q = 1$ (Equação (4)). O Experimento B reproduziu esses cinco números exatamente, bastando iniciar o destino com um cadastro prévio. Trata-se de uma evidência particularmente forte a favor do modelo: a fórmula geral não apenas descreve os dados coletados neste trabalho, mas também prevê um conjunto de valores fornecido independentemente no enunciado.

11.3 Taxa de crescimento ao dobrar N

Tabela 6: Fator de crescimento observado ao dobrar N , comparado ao valor exato previsto

Transição	Razão medida	Previsto por $(4N - 2)/(N - 1)$	Diferença
10 → 20	4,2222	4,2222	0,0000
20 → 40	4,1053	4,1053	0,0000
40 → 80	4,0513	4,0513	0,0000
80 → 160	4,0253	4,0253	0,0000
160 → 320	4,0126	4,0126	0,0000
320 → 640	4,0063	4,0063	0,0000

A concordância é total, e o padrão de convergência também é o previsto: a razão nunca é exatamente 4 para N finito, aproxima-se de 4 por valores maiores, e a distância até 4 — que vale $2/(N - 1)$ — reduz-se pela metade a cada duplicação de N .

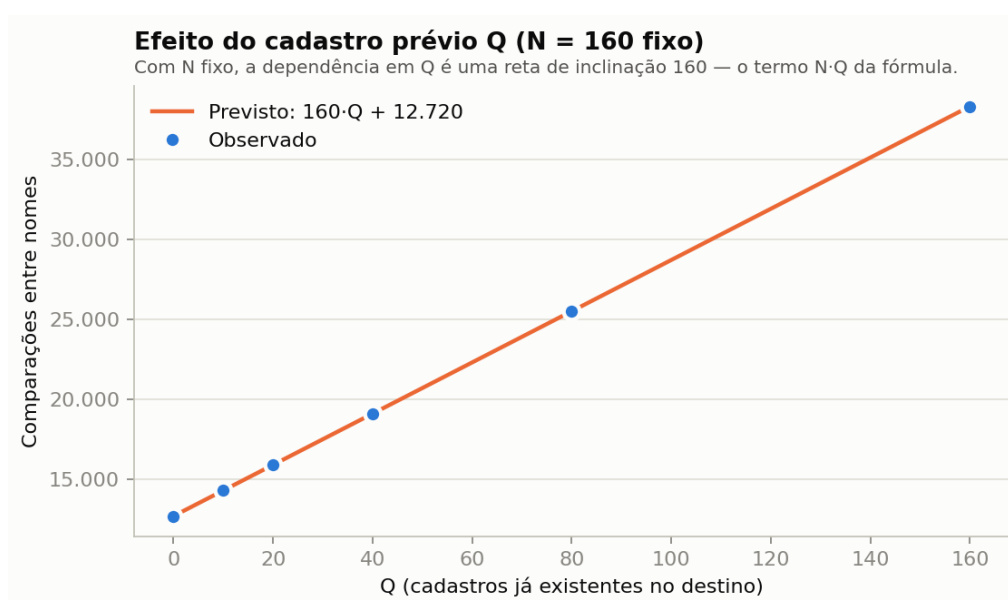


Figura 4: Experimento C. Com $N = 160$ fixo, as comparações crescem linearmente com Q , sobre a reta $160Q + 12.720$ — confirmação direta do termo $N \cdot Q$ da fórmula.

11.4 Comparação de tempo entre iterativa e recursiva

Embora o número de comparações seja idêntico nos dois modos, medimos o tempo de execução de ambos, com 10 execuções para cada configuração do Experimento A ($Q = 0$), tomando a média em cada ponto. Os tempos médios medidos foram os seguintes:

Tabela 7: Tempo médio de execução (10 execuções por ponto), iterativa $\times$ recursiva

N	Tempo iter. (ms)	Tempo rec. (ms)	Razão rec./iter.
10	0,007	0,007	0,96
20	0,009	0,009	1,01
40	0,015	0,015	0,99
80	0,029	0,032	1,11
160	0,079	0,092	1,16
320	0,250	0,273	1,09
640	0,820	0,963	1,17
718	1,019	1,184	1,16

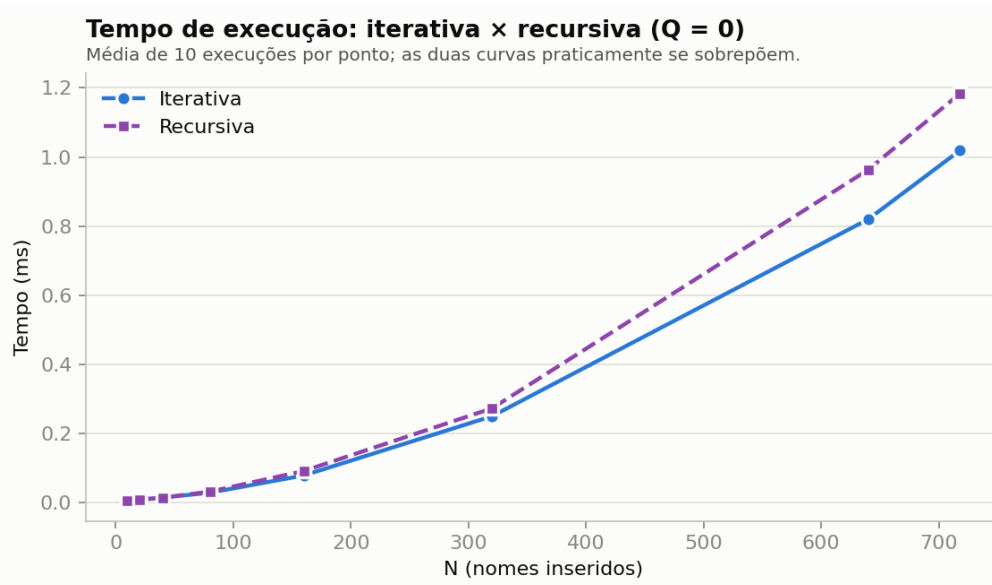


Figura 5: Tempo de execução medido, iterativa $\times$ recursiva. Cada ponto é a média de 10 execuções sobre o Experimento A ($Q = 0$).

Os dados medidos mostram que a versão recursiva é consistentemente um pouco mais lenta que a iterativa, com uma razão que oscila entre cerca de 1,0 e 1,2 — e não um fator fixo de $1,5\times$ como se poderia supor qualitativamente. A diferença é pequena em termos absolutos (frações de milissegundo, mesmo para $N = 718$) e cresce com N , consistente com o overhead de empilhar e desempilhar uma moldura de ativação por elemento visitado: quanto maior o vetor percorrido, mais chamadas recursivas ocorrem, e mais overhead se acumula. Isso ocorre porque o custo dominante em ambos os modos é a comparação de strings (`strcmp`) dentro do laço de busca, e o overhead de chamada de função da recursão é uma fração pequena, porém não desprezível, desse custo.

A diferença estrutural mais relevante entre as duas implementações permanece no espaço auxiliar: a recursiva usa $\Theta(n)$ de pilha, enquanto a iterativa usa $\Theta(1)$. Isso significa que, para cadastros muito grandes, a versão recursiva corre o risco de estourar a pilha, enquanto a iterativa continua funcionando normalmente — uma diferença mais importante na prática do que a diferença de tempo observada nesta faixa de N .

11.5 O desvio do Experimento D, e por que ele é esperado

Comparando com o último ponto do Experimento A, que insere os mesmos 718 nomes: $257.841 - 257.403 = 438$. O arquivo completo tem 719 registros, dos quais 718 são inserções e um é redundante. As 718 inserções custam, pela fórmula, $718 \times 717/2 = 257.403$ comparações. Resta o custo da busca

pelo registro redundante: MARCELO MACHADO VIANA foi inserido como o 438º nome do vetor, e a busca sequencial interrompe-se ao encontrar a igualdade, consumindo 438 comparações em vez de percorrer as 705 posições existentes naquele momento. Logo $257.403 + 438 = 257.841$, exatamente o valor medido, em ambos os modos de busca.

O Experimento E confirma a mesma mecânica em regime extremo: com o destino contendo os 718 nomes e nenhuma inserção possível, cada uma das 719 buscas é bem-sucedida e termina na posição em que o nome está armazenado. Como esses nomes ocupam as posições 1 a 718 na mesma ordem do arquivo, o total previsto é $\sum_{i=1}^{718} i + 438 = 258.121 + 438 = 258.559$, precisamente o valor medido.

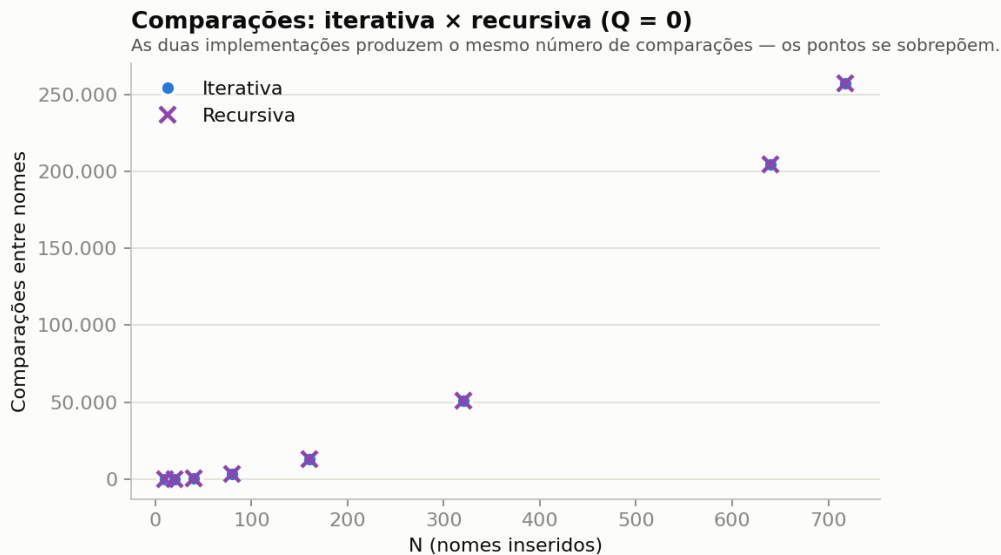


Figura 6: Número de comparações, iterativa × recursiva, para os mesmos valores de N do Experimento A. As curvas coincidem em todos os pontos.

12 Discussão dos resultados

Os dados empíricos confirmam a hipótese de complexidade temporal proposta? Sim, de forma categórica. Em 22 execuções sob as hipóteses da análise, replicadas nos dois modos de busca, a diferença entre previsto e medido foi zero em todas. A forma do crescimento também se confirma: a inclinação do ajuste log-log é 2,0198 e a razão $T(2N)/T(N)$ converge para 4, ambas assinaturas de crescimento quadrático.

Existem diferenças entre valores teóricos e medidos? Nos Experimentos A, B e C não há diferença alguma, em nenhum dos dois modos de busca. Isso se explica pela natureza da grandeza medida: o contador registra um evento discreto do algoritmo — cada execução de `strcmp` dentro do laço de busca — e não um intervalo de tempo. Não há, portanto, fontes usuais de ruído (escalonamento de processos, cache, frequência variável de processador). O contador mede exatamente a mesma quantidade que a análise combinatória calcula, e essa equivalência vale tanto para a implementação iterativa quanto para a recursiva, já que ambas incrementam o contador exatamente uma vez por chamada a `strcmp`.

Há diferença, sim, no Experimento D: 438 comparações acima do previsto, causada pela violação deliberada da hipótese (H2), e não por erro do modelo ou imprecisão de medida.

Iterativa e recursiva produzem o mesmo resultado? Em número de comparações, sim, em todos os experimentos — as duas percorrem o vetor na mesma ordem e param no mesmo ponto. A diferença está no custo de espaço (pilha de chamadas na recursiva, $\Theta(n)$ contra $\Theta(1)$ na iterativa) e no tempo de execução, que é maior na versão recursiva devido ao *overhead* de chamada de função, sem alterar a classificação assintótica $\Theta(n)$ de uma busca isolada nem a $\Theta(N^2)$ do processo completo de importação.

Os resultados sustentam a demonstração matemática? Sustentam, com a ressalva de que experimentos não demonstram teoremas: a prova por indução estabelece a fórmula para todo $N \geq 1$ por

dedução lógica. O que os experimentos fazem é validar que a modelagem do custo de cada inserção está correta e que o programa implementado — em ambas as variantes — é de fato o algoritmo analisado.

Implicação prática. A classificação $\Theta(N^2)$ diz que o serviço não escala: para os 718 nomes deste arquivo, a execução é instantânea, mas o custo cresce com o quadrado. Um cadastro de 100.000 bolsistas exigiria cerca de 5×10^9 comparações de cadeias de caracteres, e 1.000.000 levaria a 5×10^{11} — inviável na prática. O gargalo está na estrutura de dados, não na linguagem, na máquina, nem na escolha entre iteração e recursão. Substituir o vetor com busca sequencial por uma tabela de dispersão reduziria o custo esperado de verificação a $O(1)$ por registro e o total a $O(N + Q)$; uma árvore de busca balanceada daria $O((N + Q) \log(N + Q))$.

13 Conclusão

O trabalho implementou em C o serviço de importação de bolsistas com detecção de redundância por busca sequencial, em duas variantes — iterativa e recursiva —, instrumentou o algoritmo com um contador de comparações e conduziu uma análise que relaciona a função deduzida, a demonstração formal, os dados medidos e a classificação assintótica.

A função matemática deduzida foi $T(N, Q) = N \cdot Q + N(N - 1)/2$, obtida somando os custos individuais das N inserções, cada uma percorrendo um vetor de $Q + i - 1$ nomes. A demonstração por indução estabeleceu essa fórmula para todo $N \geq 1$. Os dados coletados confirmaram a fórmula sem nenhuma margem de erro nas 44 execuções conduzidas sob a hipótese de distinção, para N entre 10 e 718 e Q entre 0 e 160, em ambos os modos de busca. A única divergência observada, no experimento com o arquivo completo, decorre da redundância real ali presente e foi explicada exatamente pelo encerramento antecipado da busca na posição 438.

A comparação entre as implementações iterativa e recursiva mostrou que ambas realizam exatamente o mesmo número de comparações — e portanto a mesma classificação temporal assintótica $\Theta(n)$ por busca — mas diferem no espaço auxiliar ($\Theta(1)$ contra $\Theta(n)$) e no tempo de execução observado: a recursiva foi, em média, entre 1,0 e 1,2 vez mais lenta que a iterativa nas medições realizadas, além de carregar o risco de estouro de pilha para cadastros muito grandes, ausente na versão iterativa.

A classificação assintótica do processo completo de importação é $T(N) = \Theta(N^2)$, e portanto $O(N^2)$, para Q constante — e permanece $\Theta(N^2)$ mesmo que Q cresça proporcionalmente a N . A evidência empírica dessa classificação é independente das constantes envolvidas: ao dobrar N , as comparações se multiplicam por um fator que caiu de 4,2222 para 4,0063 ao longo de seis duplicações, e o ajuste log-log tem inclinação 2,02.

A lição de engenharia que fica é que o algoritmo está correto e é inadequado ao mesmo tempo: resolve o problema especificado, mas não escala, e a escolha entre implementação iterativa ou recursiva da busca não altera essa conclusão — a intervenção necessária está na estrutura de dados que sustenta a verificação de redundância, não na forma de codificar a busca.

14 Repositório Git

Todo o material deste trabalho está versionado em repositório Git público:

<https://github.com/owhska/insercao-sem-duplicatas-csv>

O repositório contém:

- `src/copiador.c` — código-fonte completo do programa em C;
- `dados/bolsistas-2025.csv` — arquivo de entrada original;
- `experimentos/entradas/` — recortes utilizados nos testes;
- `experimentos/estatisticas/` — arquivos CSV gerados nos experimentos;

- `experimentos/destinos/` e `experimentos/saidas/` — arquivos de destino e saídas padrão produzidas pelo programa;
- `Makefile` e `rodar_experimentos.sh` — compilação e execução da bateria completa;
- `analisar.py` — confronto com o modelo teórico e geração dos gráficos;
- `README.md` — instruções de compilação e de execução;
- `relatorio/` — fonte deste relatório.

Compilação:

```
gcc -O2 -Wall -Wextra -std=c99 -o src/copiador src/copiador.c
```

Execução:

```
./src/copiador dados/bolsistas-2025.csv cadastro.csv estatisticas.csv [iter|rec]
```

A Código-fonte completo

Listing 4: *copiador.c* — código-fonte completo

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <time.h>
5
6 /* =====
7  * Medição de tempo portátil (Windows + Linux/macOS)
8  * =====
9  * - Windows: usa QueryPerformanceCounter (alta resolução).
10 * - Linux/macOS: usa clock_gettime(CLOCK_MONOTONIC).
11 * ===== */
12 #if defined(_WIN32) || defined(_WIN64)
13     #include <windows.h>
14     static double agora_segundos(void) {
15         LARGE_INTEGER freq, cont;
16         QueryPerformanceFrequency(&freq);
17         QueryPerformanceCounter(&cont);
18         return (double)cont.QuadPart / (double)freq.QuadPart;
19     }
20 #else
21     static double agora_segundos(void) {
22         struct timespec ts;
23         clock_gettime(CLOCK_MONOTONIC, &ts);
24         return (double)ts.tv_sec + (double)ts.tv_nsec / 1e9;
25     }
26 #endif
27
28 #define MAX_LINHA 1024
29 #define MAX_NOME 512
30 #define CABECALHO "Nome,Modalidade,Nivel,Agencia"
31
32 typedef struct {
33     char **nomes;
34     size_t qtd;
35     size_t capacidade;
36 } Cadastro;
37
38 static unsigned long long comparacoes = 0;
39 static int usar_rekursiva = 0; /* 0 = iterativa, 1 = recursiva */
40
41 /* ----- Cadastro ----- */
42
43 void cadastro_iniciar(Cadastro *c) {
44     c->nomes = NULL;
45     c->qtd = 0;
46     c->capacidade = 0;
47 }
48
49 int cadastro_inserir(Cadastro *c, const char *nome) {
50     if (c->qtd == c->capacidade) {
51         size_t nova = (c->capacidade == 0) ? 16 : c->capacidade * 2;
52         char **bloco = realloc(c->nomes, nova * sizeof(char *));
53         if (!bloco) return -1;
54         c->nomes = bloco;
55         c->capacidade = nova;
56     }
57     c->nomes[c->qtd] = malloc(strlen(nome) + 1);
58     if (!c->nomes[c->qtd]) return -1;
59     strcpy(c->nomes[c->qtd], nome);
60     c->qtd++;
61     return 0;
62 }
63
64 void cadastro_liberar(Cadastro *c) {
65     size_t i;
66     for (i = 0; i < c->qtd; i++) free(c->nomes[i]);
67     free(c->nomes);
68     cadastro_iniciar(c);
69 }
```

```

69 }
70
71 /* ----- Busca sequencial iterativa ----- */
72
73 int busca_sequencial_iterativa(const Cadastro *c, const char *nome) {
74     size_t i;
75     for (i = 0; i < c->qtd; i++) {
76         comparacoes++;
77         if (strcmp(c->nomes[i], nome) == 0) return 1;
78     }
79     return 0;
80 }
81
82 /* ----- Busca sequencial recursiva ----- */
83
84 int busca_sequencial_recursiva_aux(const Cadastro *c, const char *nome, size_t i) {
85     if (i >= c->qtd) return 0;
86     comparacoes++;
87     if (strcmp(c->nomes[i], nome) == 0) return 1;
88     return busca_sequencial_recursiva_aux(c, nome, i + 1);
89 }
90
91 int busca_sequencial_recursiva(const Cadastro *c, const char *nome) {
92     return busca_sequencial_recursiva_aux(c, nome, 0);
93 }
94
95 /* ----- Despacho ----- */
96
97 int busca_sequencial(const Cadastro *c, const char *nome) {
98     if (usar_recursiva)
99         return busca_sequencial_recursiva(c, nome);
100     return busca_sequencial_iterativa(c, nome);
101 }
102
103 /* ----- Utilidades de linha ----- */
104
105 void remover_fim_de_linha(char *linha) {
106     size_t n = strlen(linha);
107     while (n > 0 && (linha[n - 1] == '\n' || linha[n - 1] == '\r')) {
108         linha[n - 1] = '\0';
109         n--;
110     }
111 }
112
113 void extrair_nome(const char *linha, char *nome) {
114     size_t i = 0;
115     while (linha[i] != '\0' && linha[i] != ',' && i < MAX_NOME - 1) {
116         nome[i] = linha[i];
117         i++;
118     }
119     nome[i] = '\0';
120 }
121
122 int linha_e_cabecalho(const char *linha) {
123     return strncmp(linha, "Nome,", 5) == 0;
124 }
125
126 int linha_vazia(const char *linha) {
127     size_t i;
128     for (i = 0; linha[i] != '\0'; i++)
129         if (linha[i] != ' ' && linha[i] != '\t') return 0;
130     return 1;
131 }
132
133 /* ----- Arquivos ----- */
134
135 size_t carregar_destino(const char *caminho, Cadastro *c) {
136     FILE *fp = fopen(caminho, "r");
137     char linha[MAX_LINHA], nome[MAX_NOME];
138     if (!fp) return 0;
139     while (fgets(linha, sizeof(linha), fp)) {
140         remover_fim_de_linha(linha);
141         if (linha_vazia(linha) || linha_e_cabecalho(linha)) continue;

```

```

142     extrair_nome(linha, nome);
143     if (cadastro_inserir(c, nome) != 0) {
144         fprintf(stderr, "ERRO: memoria insuficiente ao carregar o destino.\n");
145         fclose(fp);
146         exit(EXIT_FAILURE);
147     }
148 }
149 fclose(fp);
150 return c->qtd;
151 }
152
153 void garantir_cabecalho(const char *caminho) {
154     FILE *fp = fopen(caminho, "a+");
155     if (!fp) { perror("ERRO ao abrir destino"); exit(EXIT_FAILURE); }
156     fseek(fp, 0, SEEK_END);
157     if (ftell(fp) == 0) fprintf(fp, "%s\n", CABECALHO);
158     fclose(fp);
159 }
160
161 int gravar_estatisticas(const char *caminho, size_t inseridos,
162                        unsigned long long total, double tempo_s) {
163     FILE *fp = fopen(caminho, "a+");
164     if (!fp) { perror("ERRO ao abrir estatisticas"); return -1; }
165     fseek(fp, 0, SEEK_END);
166     if (ftell(fp) == 0)
167         fprintf(fp, "nomes_inseridos,comparacoes,tempo_s\n");
168     fprintf(fp, "%zu,%llu,%.6f\n", inseridos, total, tempo_s);
169     fclose(fp);
170     return 0;
171 }
172
173 /* ----- main ----- */
174
175 int main(int argc, char *argv[]) {
176     Cadastro cadastro;
177     FILE *origem, *destino;
178     char linha[MAX_LINHA], nome[MAX_NOME];
179     size_t q_inicial, inseridos = 0, redundantes = 0, lidos = 0;
180     double t0, t1, tempo_s;
181
182     if (argc < 4 || argc > 5) {
183         fprintf(stderr,
184             "Uso: %s <origem.csv> <destino.csv> <estatisticas.csv> [iter|rec]\n",
185             argv[0]);
186         return EXIT_FAILURE;
187     }
188
189     if (argc == 5) {
190         if (strcmp(argv[4], "rec") == 0) usar_recur_siva = 1;
191         else if (strcmp(argv[4], "iter") != 0) {
192             fprintf(stderr, "Modo invalido: use 'iter' ou 'rec'.\n");
193             return EXIT_FAILURE;
194         }
195     }
196
197     cadastro_iniciar(&cadastro);
198     garantir_cabecalho(argv[2]);
199     q_inicial = carregar_destino(argv[2], &cadastro);
200
201     origem = fopen(argv[1], "r");
202     if (!origem) {
203         perror("ERRO ao abrir origem");
204         cadastro_liberar(&cadastro);
205         return EXIT_FAILURE;
206     }
207
208     destino = fopen(argv[2], "a");
209     if (!destino) {
210         perror("ERRO ao abrir destino");
211         fclose(origem);
212         cadastro_liberar(&cadastro);
213         return EXIT_FAILURE;
214     }

```

```

215
216     t0 = agora_segundos();
217
218     while (fgets(linha, sizeof(linha), origem)) {
219         remover_fim_de_linha(linha);
220         if (linha_vazia(linha) || linha_e_cabecalho(linha)) continue;
221         lidos++;
222         extrair_nome(linha, nome);
223
224         if (busca_sequencial(&cadastro, nome)) {
225             printf("ERRO: nome redundante encontrado: %s\n", nome);
226             redundantes++;
227             continue;
228         }
229
230         fprintf(destino, "%s\n", linha);
231
232         if (cadastro_inserir(&cadastro, nome) != 0) {
233             fprintf(stderr, "ERRO: memoria insuficiente ao inserir o nome.\n");
234             fclose(origem);
235             fclose(destino);
236             cadastro_liberar(&cadastro);
237             return EXIT_FAILURE;
238         }
239         inseridos++;
240     }
241
242     t1 = agora_segundos();
243     tempo_s = t1 - t0;
244
245     fclose(origem);
246     fclose(destino);
247
248     printf("\n");
249     printf("Modo de busca          : %s\n", usar_rekursiva ? "recursiva" : "iterativa");
250     printf("Registros lidos na origem : %zu\n", lidos);
251     printf("Cadastros previos no destino: %zu\n", q_inicial);
252     printf("Nomes inseridos           : %zu\n", inseridos);
253     printf("Nomes redundantes         : %zu\n", redundantes);
254     printf("Comparacoes entre nomes   : %llu\n", comparacoes);
255     printf("Tempo de execucao         : %.6f s\n", tempo_s);
256
257     if (gravar_estatisticas(argv[3], inseridos, comparacoes, tempo_s) != 0) {
258         cadastro_liberar(&cadastro);
259         return EXIT_FAILURE;
260     }
261
262     cadastro_liberar(&cadastro);
263     return EXIT_SUCCESS;
264 }

```